Why the difference? If you recall from earlier, with JavaScript there is automatic inheritance of parent scope. The inner and outer functions have their own scope, but in JavaScript the inner scope of the anonymous function has access to the outer scope of the parent function `getCallbackWithArg()`. Not so with PHP, however. For better or worse, PHP scoping rules are simpler. The good news is that there are two mechanisms for passing data from one local scope to another when a function definition contains another function definition.

First, there's the keyword `use`, which, in the context of anonymous function declarations, provides a way to make the variables of `getCallbackWithArg()` accessible in the anonymous function's scope when `getCallbackWithArg()` executes:

```
function getCallbackWithArg(string $txt): callable
{
    return function () use ($txt): void {
        print "You gave me the string '{$txt}'" . PHP_EOL;
    };
}

getCallbackWithArg('I hope this works')(); // spoilers, it does!

Output:
You gave me the string 'I hope this works'
```

That's the first way to pass parent scope into a nested function declaration. There is another mechanism for very simple cases…

Arrow Functions

Introduced in PHP 7.4, *arrow functions* are a special type of anonymous function that have automatic access to the parent scope and always return a single expression. They can be used anywhere a long-form anonymous function can be used. The syntax of an arrow function is `fn () => EXPR`, where `EXPR` is any single valid PHP expression (requiring exactly one closing semicolon). Return type declarations go after the closing parenthesis before the `=>` arrow, and `void` isn't permitted because arrow functions *always* return.